

Array-Only Question Bank

Planned Problem

1. Reverse array (in-place and extra space)
2. Rotate array (left/right by k positions)
3. Find second largest element
4. Find missing number in range 1..n
5. Find duplicate elements
6. Merge two sorted arrays without extra space
7. Move all zeros to the end
8. Leader elements in array
9. Pair sum equals given target
10. Subarray with given sum (positive integers)
11. Kadane's algorithm (max subarray sum)
12. Equilibrium index in array
13. Majority element ($> n/2$ times)
14. Sort array of 0s, 1s, 2s (Dutch National Flag)
15. Next greater element for each array item
16. Common elements in three sorted arrays

17. Product of array elements except self
18. Find smallest positive missing number
19. Count frequency of each element (1..n range)
20. Arrange array to form largest possible number

Problem 1 – Reverse an Array

Problem:

Given an array of integers, reverse it **in-place** without using any built-in functions.

Input Format:

- First line: Integer **n** (size of array)
- Second line: **n** integers (array elements)

Output Format:

- Reversed array elements in a single line separated by spaces

Example:

Input:

5

1 2 3 4 5

Output:

5 4 3 2 1

Java Solution (Pure Array):

```
import java.util.Scanner;

public class ReverseArray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();

        int start = 0, end = n - 1;
        while (start < end) {
            int temp = arr[start];
            arr[start] = arr[end];
            arr[end] = temp;
            start++;
            end--;
        }

        for (int i = 0; i < n; i++) System.out.print(arr[i] + "
");
    }
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Problem 2 – Find Second Largest Element

Problem:

Find the second largest element in an integer array without using sorting.

Input Format:

- First line: Integer n
- Second line: n integers

Output Format:

- Single integer $\rightarrow$ second largest element

Example:

Input:

6

10 20 4 45 99 99

Output:

45

Java Solution (Pure Array):

```
import java.util.Scanner;

public class SecondLargest {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
```

```

        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();

        int first = Integer.MIN_VALUE, second =
Integer.MIN_VALUE;

        for (int i = 0; i < n; i++) {
            if (arr[i] > first) {
                second = first;
                first = arr[i];
            } else if (arr[i] > second && arr[i] != first) {
                second = arr[i];
            }
        }

        System.out.println(second);
    }
}

```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Problem 3 – Rotate Array by K Positions (Right)

Problem:

Rotate the array elements **right** by **k** positions manually.

Input Format:

- First line: Integer **n**
- Second line: **n** integers

- Third line: Integer **k** (number of rotations)

Output Format:

- Rotated array elements in single line

Example:

Input:

5

1 2 3 4 5

2

Output:

4 5 1 2 3

Java Solution (Pure Array):

```
import java.util.Scanner;

public class RotateArrayRight {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();
        int k = sc.nextInt();
        k = k % n; // to avoid extra rotations

        int[] temp = new int[n];
        for (int i = 0; i < n; i++) {
            temp[(i + k) % n] = arr[i];
        }

        for (int i = 0; i < n; i++) System.out.print(temp[i] + " ")
    }
}
```

```
");  
    }  
}
```

Time Complexity: $O(n)$

Space Complexity: $O(n)$ (extra array used)

Problem 4 – Merge Two Sorted Arrays Without Built-in Sort

Problem:

Given two sorted arrays, merge them into a single sorted array **without using sorting functions**.

Input Format:

- First line: Integer n_1 (size of first array)
- Second line: n_1 integers (sorted)
- Third line: Integer n_2 (size of second array)
- Fourth line: n_2 integers (sorted)

Output Format:

- Merged sorted array

Example:

Input:

3
1 4 7
4
2 3 6 8

Output:

1 2 3 4 6 7 8

Java Solution (Pure Array):

```
import java.util.Scanner;

public class MergeSortedArrays {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n1 = sc.nextInt();
        int[] arr1 = new int[n1];
        for (int i = 0; i < n1; i++) arr1[i] = sc.nextInt();

        int n2 = sc.nextInt();
        int[] arr2 = new int[n2];
        for (int i = 0; i < n2; i++) arr2[i] = sc.nextInt();

        int[] merged = new int[n1 + n2];
        int i = 0, j = 0, k = 0;

        while (i < n1 && j < n2) {
            if (arr1[i] <= arr2[j]) merged[k++] = arr1[i++];
            else merged[k++] = arr2[j++];
        }
        while (i < n1) merged[k++] = arr1[i++];
        while (j < n2) merged[k++] = arr2[j++];

        for (int x = 0; x < merged.length; x++)
            System.out.print(merged[x] + " ");
    }
}
```



```
}  
}
```

Time Complexity: $O(n_1 + n_2)$

Space Complexity: $O(n_1 + n_2)$

Problem 5 – Count Frequency of Each Element

Problem:

Given an array, print each element's frequency **without using collections or maps**.

Input Format:

- First line: Integer n
- Second line: n integers

Output Format:

- Each line $\rightarrow$ element: frequency

Example:

Input:

6

10 20 10 10 30 20

Output:

10: 3

20: 2

30: 1

Java Solution (Pure Array):

```
import java.util.Scanner;

public class FrequencyCount {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();

        boolean[] visited = new boolean[n];

        for (int i = 0; i < n; i++) {
            if (visited[i]) continue;
            int count = 1;
            for (int j = i + 1; j < n; j++) {
                if (arr[i] == arr[j]) {
                    visited[j] = true;
                    count++;
                }
            }
            System.out.println(arr[i] + ": " + count);
        }
    }
}
```

Time Complexity: $O(n^2)$

Space Complexity: $O(n)$ (visited array)

Problem 6 – Left Rotate Array by **k** Positions

Problem Statement:

Given an integer array and a number **k**, rotate the array elements to the left by **k** positions without using built-in rotation functions.

Input:

```
5
1 2 3 4 5
2
```

Output:

```
3 4 5 1 2
```

Example Test Cases:

Input	Output
5\n1 2 3 4 5\n2	3 4 5 1 2
4\n10 20 30 40\n3	40 10 20 30

Java Solution (Pure Array):

```
import java.util.Scanner;
class RotateLeft {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int arr[] = new int[n];
        for(int i = 0; i < n; i++) arr[i] = sc.nextInt();
        int k = sc.nextInt();
        k = k % n;
```

```

        for(int r = 0; r < k; r++) {
            int first = arr[0];
            for(int i = 0; i < n - 1; i++) {
                arr[i] = arr[i + 1];
            }
            arr[n - 1] = first;
        }

        for(int x : arr) System.out.print(x + " ");
    }
}

```

Time Complexity: $O(n*k)$

Space Complexity: $O(1)$

Problem 7 – Rearrange Positive and Negative Numbers Alternately

Problem Statement:

Given an array containing both positive and negative numbers, rearrange them alternately such that positives and negatives appear one after another. If extra positives or negatives exist, append them at the end.

Input:

6
-1 3 -5 6 -2 4

Output:

3 -1 6 -5 4 -2

Example Test Cases:

Input	Output
6\n-1 3 -5 6 -2 4	3 -1 6 -5 4 -2
5\n1 2 -3 -4 5	1 -3 2 -4 5

Java Solution (Pure Array):

```
import java.util.Scanner;
class RearrangePosNeg {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int arr[] = new int[n];
        for(int i = 0; i < n; i++) arr[i] = sc.nextInt();

        int[] pos = new int[n];
        int[] neg = new int[n];
        int p = 0, q = 0;

        for(int num : arr) {
            if(num >= 0) pos[p++] = num;
            else neg[q++] = num;
        }

        int i = 0, j = 0, k = 0;
        while(i < p && j < q) {
            arr[k++] = pos[i++];
            arr[k++] = neg[j++];
        }
        while(i < p) arr[k++] = pos[i++];
    }
}
```

```

        while(j < q) arr[k++] = neg[j++];

        for(int x : arr) System.out.print(x + " ");
    }
}

```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Problem 8 – Find Leaders in an Array

Problem Statement:

An element is a leader if it is greater than all elements to its right. The rightmost element is always a leader.

Input:

6
16 17 4 3 5 2

Output:

17 5 2

Example Test Cases:

Input	Output
6\n16 17 4 3 5 2	17 5 2
5\n1 2 3 4 0	4 0

Java Solution (Pure Array):

```
import java.util.Scanner;
class LeadersInArray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int arr[] = new int[n];
        for(int i = 0; i < n; i++) arr[i] = sc.nextInt();

        int maxFromRight = arr[n-1];
        System.out.print(maxFromRight + " ");
        for(int i = n - 2; i >= 0; i--) {
            if(arr[i] > maxFromRight) {
                maxFromRight = arr[i];
                System.out.print(maxFromRight + " ");
            }
        }
    }
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Problem 9 – Count Pairs with Given Sum

Problem Statement:

Given an integer array and a number **sum**, count the number of pairs whose sum equals **sum**.

Input:

5
1 5 7 -1 5
6

Output:

3

Example Test Cases:

Input	Output
5\n1 5 7 -1 5\n6	3
4\n1 1 1 1\n2	6

Java Solution (Pure Array):

```
import java.util.Scanner;
class PairSumCount {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int arr[] = new int[n];
        for(int i = 0; i < n; i++) arr[i] = sc.nextInt();
        int sum = sc.nextInt();

        int count = 0;
        for(int i = 0; i < n; i++) {
            for(int j = i + 1; j < n; j++) {
                if(arr[i] + arr[j] == sum) {
                    count++;
                }
            }
        }
    }
}
```



```
    }  
    System.out.println(count);  
}  
}
```

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

Problem 10: Rotate Matrix by 90 Degrees Clockwise

Problem Statement:

Given an $n \times n$ matrix, rotate it by 90 degrees clockwise **in-place**.

Input:

```
3  
1 2 3  
4 5 6  
7 8 9
```

Output:

```
7 4 1  
8 5 2  
9 6 3
```

Java Solution:

```
import java.util.Scanner;  
  
public class RotateMatrix90 {
```

```
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    int[][] matrix = new int[n][n];

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            matrix[i][j] = sc.nextInt();

    // Transpose
    for (int i = 0; i < n; i++) {
        for (int j = i; j < n; j++) {
            int temp = matrix[i][j];
            matrix[i][j] = matrix[j][i];
            matrix[j][i] = temp;
        }
    }

    // Reverse each row
    for (int i = 0; i < n; i++) {
        int left = 0, right = n - 1;
        while (left < right) {
            int temp = matrix[i][left];
            matrix[i][left] = matrix[i][right];
            matrix[i][right] = temp;
            left++;
            right--;
        }
    }

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++)
            System.out.print(matrix[i][j] + " ");
        System.out.println();
    }
}
```

```
}
```

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

Problem 11: Count Subarrays with Given Sum

Problem Statement:

Given an array and a target sum, count the number of subarrays that sum to the target.

Input:

```
5  
1 1 1 2 1  
target = 3
```

Output:

```
3
```

(The subarrays are: [1,1,1], [1,2], [2,1])

Java Solution:

```
import java.util.Scanner;  
  
public class SubarraySumCount {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        int n = sc.nextInt();
```

```

int[] arr = new int[n];
for (int i = 0; i < n; i++) arr[i] = sc.nextInt();
int target = sc.nextInt();

int count = 0;

for (int i = 0; i < n; i++) {
    int sum = 0;
    for (int j = i; j < n; j++) {
        sum += arr[j];
        if (sum == target) count++;
    }
}

System.out.println(count);
}
}

```

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

Problem 12 – Minimum Platforms Required

Description:

Given arrival and departure times of trains, find the **minimum number of platforms** required so that no train waits.

Input:

```

arrivals:    900  940  950  1100  1500  1800
departures:  910 1200 1120 1130 1900 2000

```

Output:

3

Java Solution (No Built-in Sort)

```
public class MinimumPlatforms {

    // Manual sort method (simple Bubble Sort)
    public static void bubbleSort(int[] arr) {
        int n = arr.length;
        for (int x = 0; x < n - 1; x++) {
            for (int y = 0; y < n - x - 1; y++) {
                if (arr[y] > arr[y + 1]) {
                    int temp = arr[y];
                    arr[y] = arr[y + 1];
                    arr[y + 1] = temp;
                }
            }
        }
    }

    public static int findMinPlatforms(int[] arrivals, int[]
departures) {
        int n = arrivals.length;

        // Sort both arrays manually
        bubbleSort(arrivals);
        bubbleSort(departures);

        int platformNeeded = 1;
        int maxPlatforms = 1;
        int i = 1; // pointer for arrivals
        int j = 0; // pointer for departures
```

```

        while (i < n && j < n) {
            if (arrivals[i] <= departures[j]) {
                platformNeeded++;
                i++;
                if (platformNeeded > maxPlatforms)
                    maxPlatforms = platformNeeded;
            } else {
                platformNeeded--;
                j++;
            }
        }
        return maxPlatforms;
    }

    public static void main(String[] args) {
        int arrivals[] = {900, 940, 950, 1100, 1500, 1800};
        int departures[] = {910, 1200, 1120, 1130, 1900, 2000};

        System.out.println(findMinPlatforms(arrivals,
departures)); // Output: 3
    }
}

```

Problem 13: Find Missing Number in 1..N Sequence – Using Sum Formula Method

Problem Statement:

Given a sequence of numbers from 1 to N with one number missing, find the missing number using the sum formula method.

Input:

5
1 2 4 5

Output:

3

Java Solution:

```
import java.util.Scanner;

public class MissingNumberSumFormula {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(); // N value
        int[] arr = new int[n - 1];
        int sum = 0;

        for (int i = 0; i < n - 1; i++) {
            arr[i] = sc.nextInt();
            sum += arr[i];
        }

        int totalSum = n * (n + 1) / 2;
        int missing = totalSum - sum;

        System.out.println(missing);
    }
}
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Problem 14: Replace Every Element with Next Greater Element – Last Element Becomes -1

Problem Statement:

Given an array, replace every element with the next greater element. If there is no greater element, replace with -1.

Input:

6
16 17 4 3 5 2

Output:

17 5 5 5 2 -1

Java Solution:

```
import java.util.Scanner;

public class ReplaceWithNextGreater {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];

        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();

        for (int i = 0; i < n - 1; i++) {
            int nextGreater = -1;
            for (int j = i + 1; j < n; j++) {
                if (arr[j] > arr[i]) {
                    nextGreater = arr[j];
                }
            }
            arr[i] = nextGreater;
        }
        arr[n - 1] = -1;

        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
    }
}
```



```

                break;
            }
        }
        arr[i] = nextGreater;
    }
    arr[n - 1] = -1;

    for (int val : arr) System.out.print(val + " ");
}
}

```

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

Problem 15: Wave Form Array – Sort and Swap Alternate Elements

Problem Statement:

Given an array, sort it and then rearrange in wave form.

Wave form means $arr[0] \geq arr[1] \leq arr[2] \geq arr[3] \dots$

Input:

```

6
10 90 49 2 1 5

```

Output:

```

2 1 49 10 90 5

```

Java Solution:

```
import java.util.Scanner;

public class WaveFormArray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];

        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();

        // Bubble sort
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }

        // Swap alternate elements
        for (int i = 0; i < n - 1; i += 2) {
            int temp = arr[i];
            arr[i] = arr[i + 1];
            arr[i + 1] = temp;
        }

        for (int val : arr) System.out.print(val + " ");
    }
}
```

Time Complexity: $O(n^2)$ due to bubble sort

Space Complexity: $O(1)$

Problem 16: Find Equilibrium Index

Problem Statement:

An equilibrium index is an index where the sum of elements before it is equal to the sum after it.

Input:

7
-7 1 5 2 -4 3 0

Output:

3

Java Solution:

```
import java.util.Scanner;

public class EquilibriumIndex {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();

        for (int i = 0; i < n; i++) {
            int leftSum = 0, rightSum = 0;

            for (int j = 0; j < i; j++) leftSum += arr[j];
            for (int j = i + 1; j < n; j++) rightSum += arr[j];
        }
    }
}
```

```
        if (leftSum == rightSum) {
            System.out.println(i);
            return;
        }
    }
    System.out.println(-1); // If no equilibrium index found
}
```

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$